INTERNSHIP PROJECT REPORT

IGLOO ASSET MANAGEMENT SYSTEM

Web Application + Field-Focused Mobile Application

Prepared By	Md. Sakib Al-Hasan
Internship / Department	MIS (Software Development)
Organization	Abdul Monem Ltd
Project	Igloo Asset Management System
Internship Period	17 June 2026 to 16 September 2026
Submission Date	16 September 2026

Table of Contents

SECTION	PAGE
1. Executive Summary	3
2. Internship Objectives	3
3. Project Overview	3
4. System Architecture	4
5. Technology Stack	5
6. Backend & Database Development	5
7. Database & Data Modeling	6
8. Asset Lifecycle & Core Workflows	7
9. Web Application	8
10. Mobile Application	9
11. Authentication, Authorization & Security	9
12. Role & Permission Design	11
13. Functional Requirements Implemented	11
14. Non-Functional Considerations	12
15. Testing & Validation Approach	12
16. Challenges Encountered & Problem-Solving	13
17. Learning Outcomes	14
18. Development Method & Working Approach	14
19. Future Enhancement Opportunities	14
20. Conclusion	15
21. Project Snapshot	17

1. Executive Summary

During my internship at Abdul Monem Ltd, I worked on the Igloo Asset Management System, a full-stack solution for managing freezer assets through their operational lifecycle. The project was delivered as both a web application and a lightweight mobile application, connected to a common REST API and MongoDB database.

The system provides structured asset registration, unique identification, authorization, outlet assignment, transfer, return, repair, physical verification, check-in, disposal, request management, master-data management, and user access control. The mobile application focuses on field activities such as asset search, scanning, verification, check-in, repair requests, and asset operations.

The project gave me practical experience in connecting business requirements with software architecture, database design, API development, authentication, authorization, responsive web development, mobile development, debugging, validation, and integration.

2. Internship Objectives

- Understand the complete lifecycle of a real-world software project from requirement interpretation to implementation and refinement.
- Develop practical full-stack skills using Node.js, Express, MongoDB/Mongoose, React, and REST APIs.
- Learn how to model business entities and relationships for an asset-oriented operational system.
- Implement secure authentication and authorization using JWT, bcrypt, role-based access, and permission-aware UI/API behavior.
- Build responsive web workflows for administrators and operational users.
- Develop a lightweight mobile application for field-oriented activities using React Native and Expo.
- Practice debugging, integration, validation, environment configuration, and iterative delivery.
- Improve professional skills in problem decomposition, code organization, documentation, and communicating technical work to non-technical stakeholders.

3. Project Overview

The system is centered on deep freezer asset management. The core business challenge is maintaining a trustworthy record of what an asset is, where it is, who is responsible for it, whether it has been authorized and verified, and what operational events have happened during its lifecycle.

The solution introduces a centralized digital record for each asset. The Asset model includes an Asset Identification Number (AIN), serial number, capacity, manufacturing month and year, manufacturer, authorization reference, current outlet, lifecycle status, verification status, activation information, and timestamps. This provides a foundation for traceability rather than relying on disconnected manual records.

Core Scope

- Asset registration and identification

- Authorization and approval workflows
- Outlet mapping and asset assignment
- Asset transfer, return, repair, disposal, and history
- Physical verification and verification mapping
- Outlet check-in with GPS latitude/longitude
- Asset requests and repair requests
- User management and role/permission control
- Region, district, distributor, and manufacturer master data
- Password recovery and profile management
- Mobile field workflows including search, scanning, verification, check-in, repairs, and operations

4. System Architecture

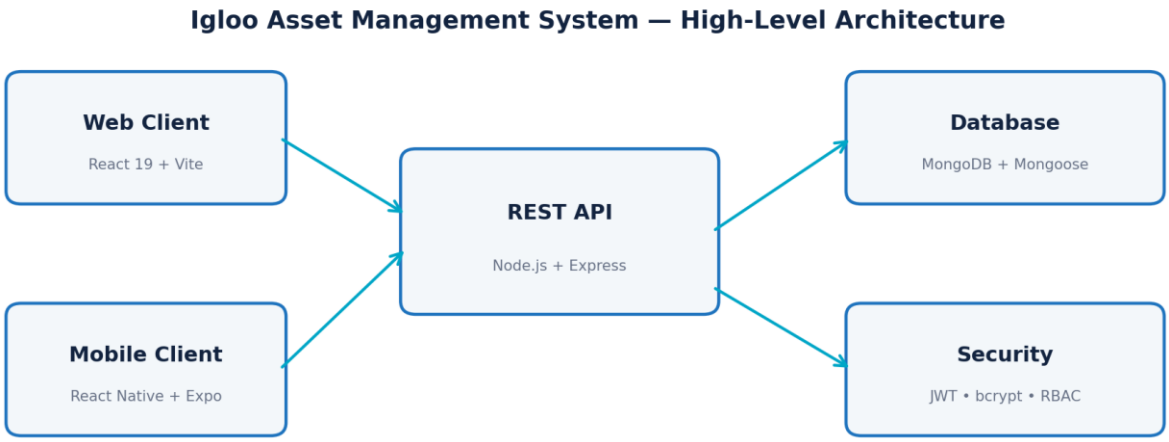


Figure 1. Igloo Asset Management System High-level architecture

The solution follows a client–API–database architecture. The web client is implemented with React and Vite, while the mobile client uses React Native with Expo and TypeScript. Both clients communicate with the Node.js/Express backend through REST endpoints. MongoDB is accessed through Mongoose models, while authentication and authorization are enforced through middleware and permission-aware application logic.

Main Layers

Presentation layer: React web pages, responsive styling, navigation, notifications, protected routes, and React Native/Expo mobile screens.

Application/API layer: Express routes and controllers implementing authentication, assets, requests, repairs, authorizations, verification, check-ins, master data, and user operations.

Business/data layer: Mongoose schemas and service utilities for asset identification, sequence generation, permission synchronization, and email-based password recovery.

Security layer: JWT authentication, bcrypt password hashing/comparison, role checks, granular permissions, protected routes, inactive-account handling, and token cleanup.

Persistence layer: MongoDB collections represented by models such as User, Asset, Outlet, AssetRequest, Authorization, Repair, VerificationMapping, CheckIn, Disposal, and master-data models.

5. Technology Stack

Layer	Technology	Practical Purpose
Backend	Node.js + Express 5	REST API, routing, controllers, middleware, error handling
Database	MongoDB + Mongoose 9	Document storage, schemas, references, indexes, timestamps
Authentication	JWT + bcrypt	Stateless authentication and password security
Email	Resend	Password-reset email delivery
Uploads	Multer	Profile-picture upload handling
Web frontend	React 19 + Vite	Component-based web interface and development/build workflow
Web HTTP	Axios	Centralized API client with token injection and 401 handling
Web routing	React Router	Application navigation and protected page routing
Mobile	React Native + Expo 57	Cross-platform field application
Mobile navigation	Expo Router	File-based navigation and screen organization
Mobile security	Expo SecureStore	Secure persistence of authentication token and configurable API URL
Mobile device features	Expo Camera / Location	QR scanning and location-aware field operations
Language mix	JavaScript, JSX, TypeScript, TSX	Backend/web development and strongly typed mobile code

6. Backend & API Development

Express REST API

The backend exposes REST endpoints grouped by business domain. The application mounts routes for authentication, users, outlets, manufacturers, authorizations, assets, asset history, repairs, disposals, check-

ins, verification mappings, asset requests, regions, districts, and distributors. This separation keeps route definitions focused on HTTP concerns while controllers handle business operations.

API Domain	Purpose
/api/auth	Login, profile, change password, forgot/reset password
/api/users	User management and access control
/api/assets	Asset registration and lifecycle operations
/api/asset-history	Historical asset events and traceability
/api/asset-requests	Requests for asset quantities and approval
/api/authorizations	Asset authorization workflow
/api/verification-mappings	Physical asset-to-outlet verification workflow
/api/repairs	Repair request and lifecycle management
/api/disposals	Controlled asset disposal
/api/check-ins	Office/outlet check-in and history
/api/outlets	Outlet management
/api/regions, /api/districts	Geographical master data
/api/distributors, /api/manufacturers	Partner/manufacturer master data

Reusable Services

- The AIN service formats capacity, manufacturing month/year, manufacturer code, and serial information into a structured Asset Identification Number.
- The counter service uses an atomic MongoDB find-and-update operation to generate sequential values safely for configured keys.
- The permission service maintains default access profiles for operational roles and synchronizes those permissions for existing users.
- The email service integrates password recovery with a transactional email provider.
- Central API handling in the web and mobile clients reduces repeated authentication and error-handling code.

7. Database & Data Modeling

The database design uses MongoDB documents with Mongoose schemas. Relationships between operational records are represented through ObjectId references. This approach keeps each domain entity explicit while allowing related information to be populated or queried through references.

Entity	Key Responsibility	Important Data
User	Identity, role, permissions, account status	name, email, password, role, permissions, assigned relationships
Asset	Central asset record	AIN, serial number, capacity, manufacture date, manufacturer, outlet, status, verification
Outlet	Physical/customer location	name, code, type, address, district, distributor, contact, phone, status
Authorization	Approval gate before/around asset use	asset type, capacity, manufacture info, outlet, requester, approver, status
AssetRequest	Request for one or more assets	outlet, requester, ASO, quantity, reason, status, review data
Repair	Maintenance lifecycle	asset, reporter, ASO, problem, service provider, cost, status, dates
VerificationMapping	Physical verification and mapping	asset, outlet, submitter, reviewer, status, remarks
CheckIn	Field/office attendance evidence	user, type, outlet, latitude, longitude, time, remarks
Disposal	Asset retirement record	asset, disposer, reason, disposal date, remarks
AssetHistory	Traceability of lifecycle events	asset and event-specific historical information
Region / District / Distributor / Manufacturer	Master data hierarchy and references	names, codes/statuses and relationships

8. Asset Lifecycle & Core Workflows

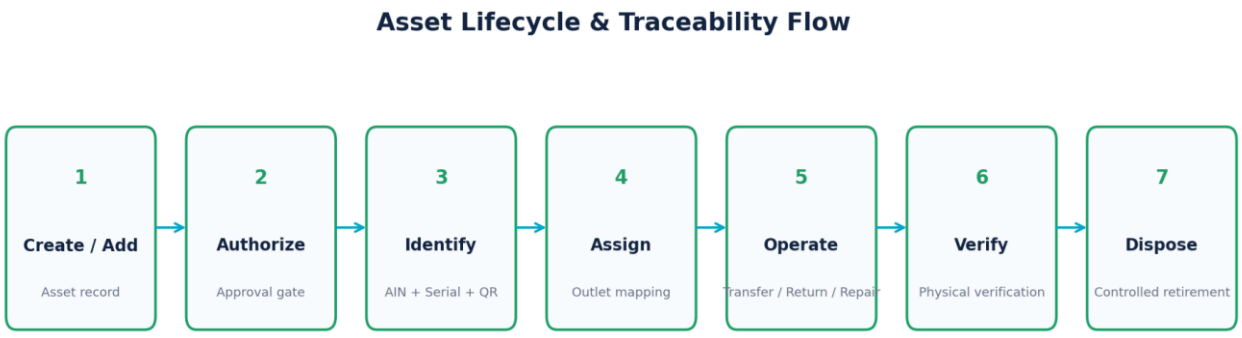


Figure 2. Conceptual asset lifecycle implemented across the web and mobile experiences.

Asset creation: A new asset can be registered with manufacturer and manufacturing information. The system maintains unique AIN and serial identifiers.

Authorization: Authorization records contain asset type, technical information, requester, assigned reviewer, status, decision date, and rejection reason.

Assignment and transfer: Assets can be assigned to an outlet and later transferred or returned, maintaining the current outlet and lifecycle status.

Verification mapping: A field user can identify an asset and map it to an outlet. The mapping remains pending until reviewed; rejection can carry remarks for correction and resubmission.

Repair: A repair record links directly to an asset and captures the reported problem, service provider, remarks, cost, status, and completion/rejection information.

Check-in: The check-in model records whether the event is office or outlet based and stores latitude, longitude, time, and optional remarks.

Disposal: Disposal is represented as a dedicated record tied to the asset, including reason, date, user, and remarks.

9. Web Application

The web application is built as a modular React interface. The submitted source contains dedicated pages for authentication, dashboard, assets, asset details/history/QR, assignment, transfer, return, disposal, requests, authorizations, verification mapping/history, repairs, check-ins, outlets, regions, districts, distributors, manufacturers, users, access rights, profiles, and password recovery.

Web Application Strengths

- Centralized API client using Axios with automatic Bearer-token attachment.
- Automatic local session cleanup when the backend returns HTTP 401.
- Protected routing and permission-aware navigation.
- Reusable layout components such as sidebar, navbar, page layout, searchable selects, and notifications.
- Separate JSX and CSS organization for maintainability.
- Operational tables and forms designed around the actual asset-management workflow rather than generic CRUD screens.
- Dedicated Access Rights configuration supports granular permissions without adding unnecessary delete operations.

Key Web Modules

Module	Purpose
Dashboard	Operational overview and navigation entry point.
Assets	View, add, add existing, inspect details, history, QR, and lifecycle operations.
Requests	Create, view, and approve/reject asset requests.
Authorization	Review and decide authorization records.

Verification Mapping	Submit, review, approve/reject, and inspect history.
Repairs	Create and manage repair requests and their lifecycle.
Check-in	Record field/office attendance and review history.
Master Data	Manage regions, districts, distributors, manufacturers, and outlets.
Users & Access Rights	Manage users and configure role/permission access.
Profile & Security	Profile management, password change, forgot password, and reset password.

10. Mobile Application

The mobile application was implemented with React Native, Expo, TypeScript, Expo Router, Expo Camera, Expo Location, and SecureStore. Rather than reproducing every administrative web feature, the mobile app focuses on field workflows where mobility provides the greatest value.

Implemented Mobile Screens

- Login, session restoration, logout, profile, forgot password, and reset password.
- Dashboard and tab-based navigation.
- Asset search, asset details, asset history, add asset, and add existing asset.
- QR/barcode-oriented scan workflow.
- Asset assignment, transfer, return, and disposal operations.
- Outlet viewing and field-oriented outlet interaction.
- Verification mapping workflow.
- Check-in workflow.
- Repair request creation and repair request history.

Mobile Engineering Learning

- Using TypeScript interfaces to define API-facing asset, request, outlet, region, district, distributor, repair, and check-in data structures.
- Handling asynchronous API calls and explicit API error objects.
- Persisting the JWT securely through Expo SecureStore rather than ordinary browser localStorage.
- Detecting a development host and allowing a custom API URL for device testing.
- Integrating camera and location capabilities into business workflows.
- Using Expo Router's file-based navigation to keep screens organized.

11. Authentication, Authorization & Security

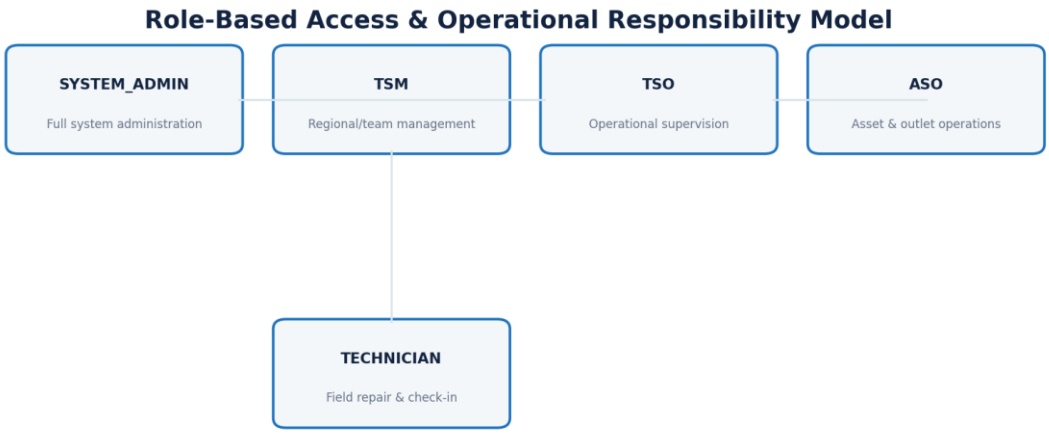


Figure 3. Role model represented by the submitted permission architecture.

The User model defines five operational roles: SYSTEM_ADMIN, ASO, TSO, TSM, and TECHNICIAN. In addition to role-level checks, the system contains granular permission groups covering dashboards, users, assets, asset requests, operations, repairs, authorization, verification mapping, outlets, check-ins, regions, districts, distributors, manufacturers, and reports.

Security Controls Observed

Control	Implementation Learning
Password protection	Passwords are handled with bcrypt rather than being stored as plain text.
JWT authentication	Successful login issues a JWT containing user identity/role information and protected endpoints use Bearer authentication.
Inactive-account protection	Login and application session logic reject inactive users.
Role-based access	Backend middleware and application logic distinguish roles and their permitted responsibilities.
Granular permissions	Non-admin roles can be evaluated at permission-group and nested-action level.
Password reset security	Reset tokens are cryptographically generated, stored in hashed form, expire after a limited period, and are invalidated after use.
CORS control	The backend explicitly manages allowed origins and local development origins.
Client session cleanup	Web and mobile clients remove stored authentication state when a 401 response indicates an invalid/expired session.
Secure mobile storage	Mobile authentication tokens are stored with Expo SecureStore.

Security mindset learned

Security is not a single login screen. It has to be considered at credential storage, token handling, route protection, role/permission checks, account status, reset flows, CORS, and client-side session behavior.

12. Role & Permission Design

Capability	SYSTEM_ADMIN	TSM	TSO	ASO	TECHNICIAN
Dashboard	Full	View	View	View	View
Users	Manage	View/Add/Status	Scoped operational access	Scoped operational access	—
Assets	Full	View/Add/Existing	View/Add/Existing	View/Add/Existing	View
Asset Requests	Full	Create/Approve	Create / workflow	Create / workflow	—
Assignment / Transfer / Return / Disposal	Full	All	Operational	Transfer	—
Repairs	Full	Manage	Operational	Operational	View/Create/Update
Authorization	Full	View/Approve	View/Approve	View	—
Verification	Full	Manage	Operational	Submit / View	—
Outlets	Full	Manage	Operational	View/Add/Update	View
Check-in	Full	All history	History	Check-in/History	Check-in/History
Master Data	Full	Manage	Scoped	View/Add/Update	View

The exact effective access for a user is determined by the permission object stored with the user and by backend authorization logic. SYSTEM_ADMIN is treated as having full permission in the application-level permission helper, while operational roles receive predefined access profiles that can be synchronized.

13. Functional Requirements Implemented

Requirement Area	Implemented Capability
Identity & access	Users can authenticate, maintain profiles, change passwords, recover passwords, and operate according to role/permission.
Asset management	Users can create, view, inspect, identify, assign, transfer, return, repair, verify, and dispose of assets.
Authorization	Assets/asset-related requests can move through approval and rejection decisions with reviewer information.
Requests	Operational users can create asset requests and designated users can review them.
Verification	Field users can submit asset-to-outlet verification mappings; reviewers can approve/reject with remarks.

Outlet management	Outlets include hierarchical location references, contact information, codes, and active/inactive status.
Check-in	The system records office/outlet check-ins with GPS coordinates and timestamps.
Master data	Regions, districts, distributors, and manufacturers support the asset/outlet hierarchy.
Mobile operations	Field users can access a focused set of asset, verification, repair, scan, check-in, and operational workflows from a mobile device.

14. Non-Functional Considerations

Maintainability: The backend separates routes, controllers, models, middleware, services, and utilities. The frontend separates pages, components, services, context, utilities, and styling.

Security: Authentication, password hashing, token handling, permissions, account status, CORS, and secure mobile storage are incorporated.

Usability: The web app uses reusable layouts and searchable controls; the mobile app concentrates on field-relevant actions and uses device capabilities.

Scalability: Domain-based routes/models and indexed request fields provide a foundation for expanding operational volume and features.

Traceability: Asset history, verification status, approval records, lifecycle statuses, and timestamps create an auditable operational trail.

Deployment awareness: The backend binds to 0.0.0.0 for LAN/device access during development, and the mobile client includes API URL detection/configuration support.

Professional engineering lesson

A feature is not complete simply because a screen works. Maintainability, security, traceability, error handling, and integration behavior are part of a production-oriented mindset.

15. Testing & Validation Approach

The submitted repository does not contain a dedicated automated unit/integration test suite or a test runner script. Therefore, this report does not claim automated test coverage. The practical validation approach for the project is best described as manual functional and integration testing across API endpoints, web workflows, mobile workflows, authentication states, and role/permission scenarios.

Recommended Functional Test Matrix

Area	Scenario	Expected Result
Authentication	Valid login / invalid credentials / inactive user / expired session	Access is granted or rejected according to state
Authorization	Allowed role vs disallowed role	Protected operation succeeds only for permitted users
Asset creation	Required fields / duplicate identifiers / invalid values	Validation and uniqueness rules are enforced
Lifecycle	Assign → transfer → return → repair → dispose	Status/location/history remain consistent
Verification	Submit → approve / reject → resubmit	Review state and remarks behave correctly
Check-in	Office/outlet + GPS values	Check-in record stores type, coordinates, and time
Password reset	Request → token → reset → reuse token	Token expires and becomes invalid after successful reset
Mobile session	Login → close/reopen → restore → logout	Secure token persistence and cleanup behave correctly
API connectivity	Valid server / unavailable server / timeout	User receives controlled connection errors

For a future production release, automated unit tests for services/controllers, integration tests for critical workflows, API contract tests, and end-to-end tests for high-risk approval and lifecycle flows would be valuable additions.

16. Challenges Encountered & Problem-Solving

Understanding business rules: Asset management contains many state transitions and approval relationships. The solution was to translate each workflow into explicit models, statuses, controllers, and UI actions.

Maintaining consistency across web and mobile: The mobile app had to consume the same backend while presenting a smaller field-focused experience. A shared REST API and dedicated API service layers helped keep integration consistent.

Authentication across platforms: Browser localStorage and mobile SecureStore have different capabilities. Separate client-side token handling was implemented while keeping the backend JWT contract consistent.

Role and permission complexity: Different users need different combinations of view/create/update/approve actions. A nested permission model and permission helper made this more manageable.

Network/device development: Mobile testing requires the device to reach the development machine. The app includes debugger-host detection, fallbacks, and configurable API URL support.

Iterative debugging: Real development involves resolving import issues, API mismatches, routing behavior, validation problems, and UI integration defects. The project provided repeated practice in tracing a problem across client, API, and database layers.

17. Learning Outcomes

Backend Engineering

- Built REST APIs with Node.js and Express.
- Learned separation of routes, controllers, middleware, models, and services.
- Worked with MongoDB and Mongoose schemas, references, timestamps, indexes, and update operations.
- Implemented authentication using JWT and password hashing using bcrypt.
- Learned validation, error handling, CORS, environment configuration, and API integration.

Frontend Engineering

- Built modular React pages and reusable components.
- Used React Router for navigation and protected routes.
- Used Axios interceptors for centralized token handling and session cleanup.
- Managed UI state, forms, notifications, permission-aware rendering, and responsive layouts.
- Learned to keep UI structure and business workflow aligned.

Mobile Development

- Built React Native screens with Expo and Expo Router.
- Used TypeScript for API models and application context.
- Integrated QR scanning/camera and location capabilities.
- Stored authentication tokens securely with Expo SecureStore.
- Designed a focused mobile workflow instead of copying the entire web system.

Security & Architecture

- Understood the difference between authentication and authorization.
- Implemented role-based and granular permission concepts.
- Learned why password reset tokens should be temporary, protected, and invalidated.
- Understood why security must exist at both frontend and backend layers.
- Learned how clients, APIs, business logic, and databases fit together.

Professional Skills

- Requirement interpretation and business-process decomposition.
- Debugging across multiple application layers.
- Incremental implementation and validation.
- Communicating technical progress and design decisions.
- Writing maintainable code rather than only code that works once.

18. Development Method & Working Approach

Requirement-first thinking: Business workflows were considered before implementing individual screens. This reduced the risk of building disconnected features.

Incremental development: The system evolved through focused feature implementation, debugging, validation, and refinement rather than attempting every module at once.

Source-of-truth discipline: The backend API and data model act as the central contract used by both web and mobile clients.

Reusable components and services: Shared UI components, API clients, middleware, and service utilities reduced duplication.

Field-focused product thinking: The mobile application intentionally emphasizes activities that benefit from mobility instead of reproducing every administrative web feature.

19. Future Enhancement Opportunities

- Automated unit, integration, and end-to-end test coverage for critical workflows.
- Production-grade logging, monitoring, alerting, and audit dashboards.
- Advanced reporting and analytics for asset distribution, verification, repair cost, and lifecycle trends.
- Offline-first mobile workflows with queued synchronization for low-connectivity field areas.
- Push notifications for approvals, rejected verification mappings, repair status, and requests.
- More comprehensive asset scanning and label-printing workflows.
- Advanced search/filtering and export capabilities for operational reporting.
- Deployment hardening with HTTPS, secrets management, environment-specific configuration, and CI/CD.
- Database backup/recovery strategy and production indexing/performance review.
- Formal API documentation using OpenAPI/Swagger and stronger automated validation.

These are presented as future opportunities rather than claims that they are already implemented in the submitted version.

20. Conclusion

The Igloo Asset Management System was a valuable practical software-engineering project because it required the integration of business rules, data modeling, APIs, security, user interfaces, mobile development, and operational workflows. The project moved my learning beyond isolated tutorials and into the realities of maintaining consistency across multiple application layers.

From a technical perspective, the internship strengthened my understanding of full-stack development: how a request moves from a UI form to an authenticated API endpoint, through business logic and database operations, and back to a meaningful user experience. From a professional perspective, it improved my ability

to break down requirements, debug systematically, preserve existing workflows, and think about access control and maintainability.

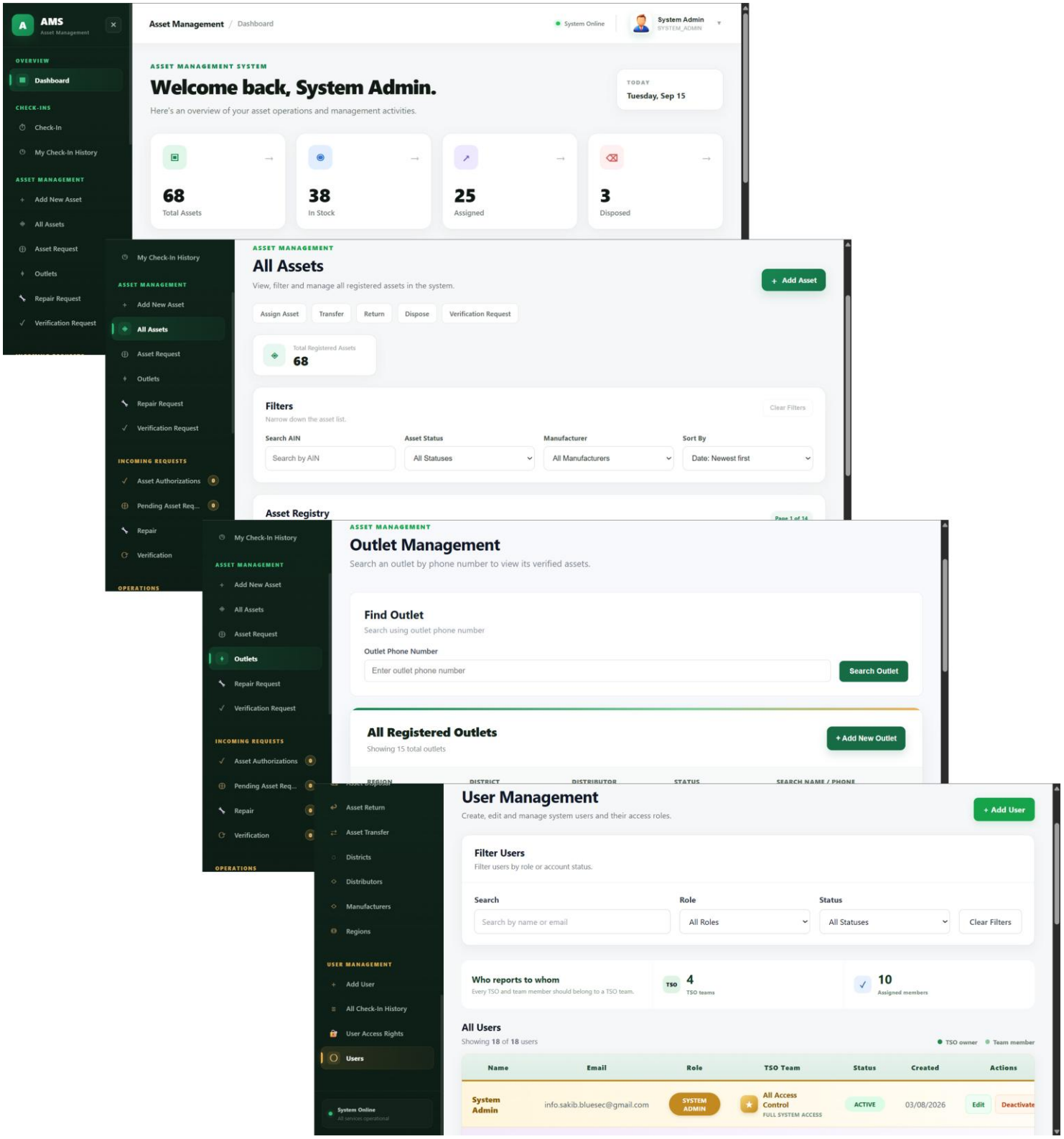
The most important lesson I gained is that software development is not only about writing code. It is about understanding a problem, designing a reliable solution, protecting it, validating it, and delivering an experience that users can actually operate. The completed web and mobile applications represent the practical application of that learning during my internship.

Closing statement

This internship has been more than just a professional experience; it has been a journey of learning, growth, and self-discovery. Building the Igloo Asset Management System taught me how to turn challenges into solutions and ideas into something meaningful. I am grateful for the guidance, trust, and opportunities that helped me grow throughout this journey. **I may be closing this chapter today, but I carry forward the knowledge, confidence, and determination to build an even better tomorrow.**

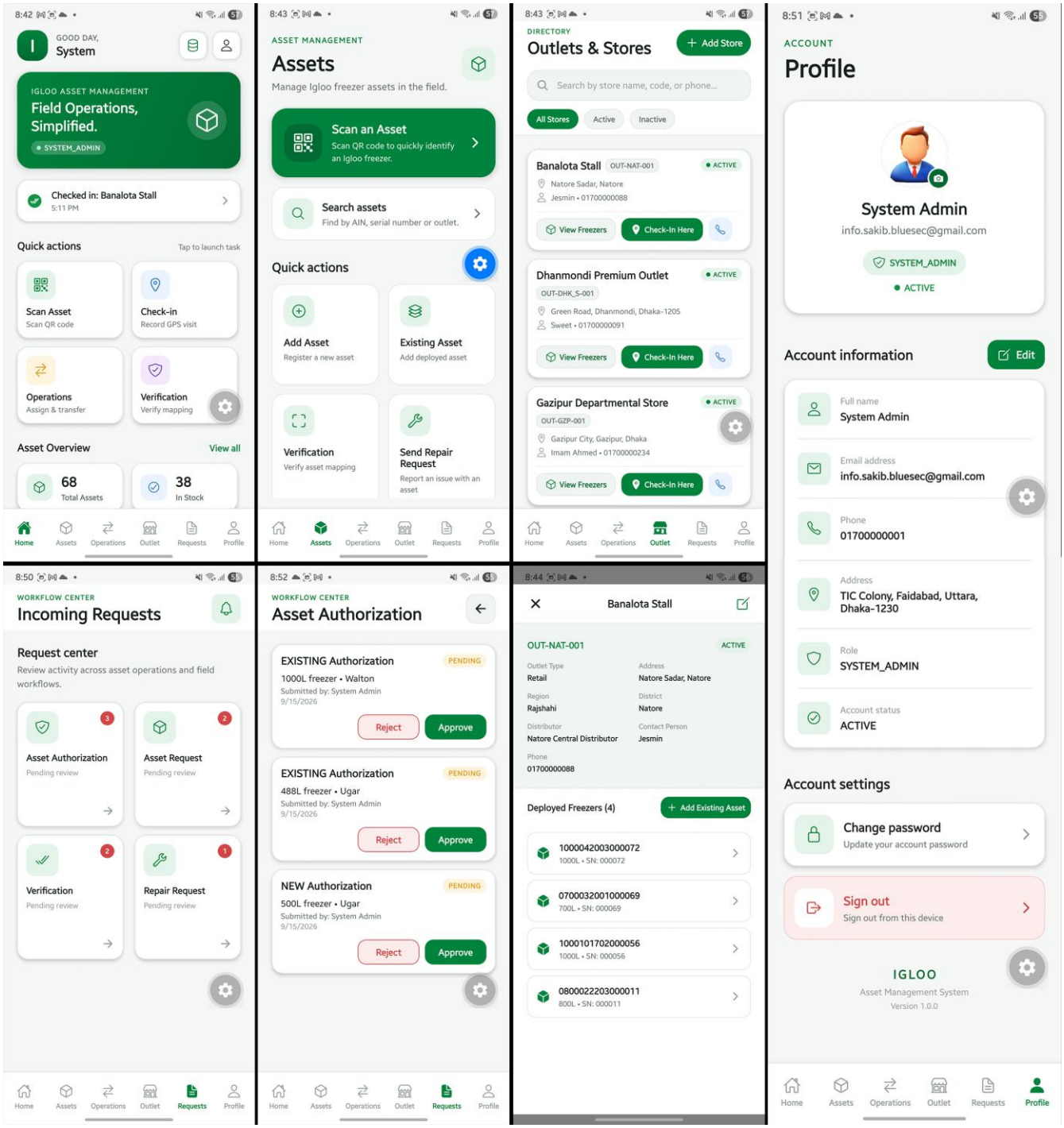
Project Snapshot

Web App:



Snap 1. A few shots of Web Application.

Mobile App:



Snap 1. A few shots of Mobile Application.

Submission Checklist

Md. Sakib Al-Hasan
Intern
MIS (Software Development)
Abdul Monem Ltd
Monem Business District, Level 13,
111 Bir Uttam C.R. Dutta Road (Sonargaon Road),
Karwan Bazar, Dhaka-1205, Bangladesh

Intern / Developer	Supervisor
Signature: _____	Signature: _____
Name: Md. Sakib Al-Hasan	Name: F.M Zaidul Haque
Date: 16 September 2026	Date: 16 September 2026